

1 Synthesis Corpus Construction and Reproducibility Protocol

This appendix documents corpus construction, selection, transition budgets, and provenance supporting **Section 2.2**, together with validation and artifact-freezing details supporting **Section 2.3**.

1.1 Corpus Sources, Labels, and Partitioning Protocol

Corpora derive from SPI-DB* and PatchDB*, retaining benchmark-provided training, development, and test partitions. The split unit is the commit patch; multi-file edits within a commit remain intact. Datasets are processed independently without pooling examples across datasets. Upstream deduplication ensures no identical commit transitions cross partition boundaries. Let $\mathcal{T}_w^{\text{pool}}$ denote the eligible weakness-specific training pool before selection, with $\mathcal{T}_w \subseteq \mathcal{T}_w^{\text{pool}}$.

Binary supervision roles are defined as follows. The positive class $(\mathcal{T}_w^+, \mathcal{V}_w^+)$ comprises security-labeled changes mapped to target weakness w via published vulnerability-to-CWE mappings. The negative class $(\mathcal{T}_w^-, \mathcal{V}_w^-)$ comprises non-security-labeled changes drawn from training and development partitions, respectively. These labels provide negative supervision without establishing the absence of w . The complete development partition is held out from synthesis selection; each subset $\mathcal{V}_w$ is constructed exclusively from that partition, guaranteeing $\mathcal{T}_w^{\text{pool}} \cap \mathcal{V}_w = \emptyset$ and $(\mathcal{T}_w^{\text{pool}} \cup \mathcal{V}_w) \cap \text{Test} = \emptyset$.

1.2 CWE Mappings, Hierarchy, and Omission Handling

Weakness assignments use bundled vulnerability-to-CWE mappings and MITRE CWE taxonomy version 4.13, applying exact identifier matching without hierarchy expansion. Security-labeled examples mapped only to other weaknesses are excluded from the target suite’s construction corpus. Handling of atypical instances follows strict accounting. For unmapped annotations, security-labeled examples lacking usable CWE mappings are excluded from positive synthesis and cataloged in `manifest_unmapped.jsonl`, while AST extraction failures are recorded independently as `PARSE_FAILURE`. For evaluation integrity, benchmark test partitions retain all test instances. Ground-truth CWE annotations are never used to determine verification outcomes. Missing or unanalyzable evidence leaves affected obligations `UNRESOLVED`; the decision module assigns the final outcome via frozen aggregation rules.

1.3 Stratification and Deterministic Example Selection

Selection for synthesis corpus $\mathcal{T}_w$ is stratified strictly by class ($y \in \{0, 1\}$) with a quota cap of $N_{\text{max}} = 20$ instances per class. Commit author timestamps are converted to UTC Unix epoch seconds (t_c), where instances with missing or non-parseable author timestamps are assigned $t_c = 0$. Within each class, eligible examples are sorted deterministically by (t_c, h_c) , where h_c is the full hexadecimal commit hash. The first $\min(n, 20)$ examples are selected, where n is the eligible class count.

Example-specific CVE identifiers, advisories, commit hashes, repository identifiers, author metadata, and benchmark annotations are excluded from synthesis prompts, except for example-role labels. Code identifiers and the target CWE specification remain available. Selection provenance is archived in `manifest_selected_examples.jsonl`.

1.4 Transition Construction, Budgets, and Truncation Rules

Each admitted commit–parent comparison yields a transition $x_i = (S_i^-, \Delta_i, S_i^+)$ subject to fixed budgets. Pre- and post-patch contexts (S_i^-, S_i^+) are capped at 8,000 Unicode code points each, traversed depth-first up to 10,000 AST nodes per state; contexts exceeding 8,000 code points truncate at statement boundaries, retaining function headers and enclosing signatures.

Parent-relative unified diffs (Δ_i) are capped at 16,000 Unicode code points, admitting complete hunks in their original order until the budget is exhausted, or truncating at the line level if an initial hunk alone exceeds 16,000 code points. Truncation metadata (TRUNCATED_CONTEXT, TRUNCATED_DIFF) are logged externally without corrupting source syntax, and obligations requiring omitted content evaluate to UNRESOLVED.

1.5 Multi-CWE Assignments and Cross-Suite Reuse

Multi-CWE patches supply positive candidates independently to each corresponding target suite. Non-security training commits originate from a shared negative pool and may appear across multiple synthesis suites ($\mathcal{T}_{w_a}^-$ and $\mathcal{T}_{w_b}^-$).

1.6 Per-Suite Training and Development Statistics

Table 1 reports per-suite counts; complete listings reside in `manifest_suite_counts.jsonl`. Development subsets are selected independently of the synthesis quota. Development positives comprise all eligible repairs mapped to w in the development partition. Development negatives follow the same ordering rules, with $|\mathcal{V}_w^-| = \min(n_w^-, |\mathcal{V}_w^+|)$, where n_w^- is the eligible development-negative count. In SPI-DB*, eligible training negatives comprise the complete set of 9,202 non-security-labeled training changes. In PatchDB*, the 14,337 eligible negatives exclude 1,621 commits modifying documentation, build scripts, and metadata outside the supported source-analysis scope. Each weakness-specific scorer is fitted on distinct candidate-weakness pairs from $\mathcal{T}_w$. Development loss on $\mathcal{V}_w$ selects the regularization strength and stopping point, as described in Section 2.3.

Table 1: Dataset-separated instance accounting across representative synthesis ($\mathcal{T}_w$) and balanced development ($\mathcal{V}_w$) partitions.

Dataset	Weakness ID (w)	Eligible Training		Synthesis ($\mathcal{T}_w$)		Development ($\mathcal{V}_w$)	
		Pos (+)	Neg (−)	Pos (+)	Neg (−)	Pos (+)	Neg (−)
PatchDB*	CWE-119 (Memory Buffer)	412	14,337	20	20	52	52
	CWE-125 (Out-of-bounds Read)	348	14,337	20	20	44	44
	CWE-787 (Out-of-bounds Write)	295	14,337	20	20	38	38
SPI-DB*	CWE-119 (Memory Buffer)	284	9,202	20	20	36	36
	CWE-125 (Out-of-bounds Read)	210	9,202	20	20	28	28
	CWE-787 (Out-of-bounds Write)	186	9,202	20	20	24	24

1.7 Suite Acceptance, Freezing, and Replication Procedure

Candidate suites are evaluated on $\mathcal{V}_w$ at the whole-suite decision level. Acceptance requires precision $p \geq 0.80$, recall $r \geq 0.80$, and FPR $f \leq 0.10$. VERIFIED represents the positive prediction; REJECTED and INCONCLUSIVE represent non-positive predictions. No validation example is excluded from metric computation. Suites with fewer than 10 actual validation positives are marked unsupported. When no positive prediction is produced, precision is assigned zero for acceptance checking. If an actual-class denominator is zero, the associated rate (recall or FPR) is undefined and the candidate fails acceptance. Candidates undergo an initial evaluation followed by at most three revisions ($r \leq 3$); failing suites are marked unsupported. Accepted artifacts package the verifier suite, schema, operator implementations, evidence validator, decision rules, feature extractor, and fitted scorer. Canonical serialization records their SHA-256 integrity digest before test evaluation.

1.8 Development Membership, Sampling, and Truncation Records

Each weakness-specific development subset $\mathcal{V}_w$ follows the annotation and class-assignment policies in Sections 1.1 and 1.2, and the selection procedure in Section 1.6. Test instances remain inaccessible throughout artifact construction.

The replication package archives development records for every suite in each dataset: Membership and selection: `manifest_development_membership.jsonl` records eligible candidates, selected members, and excluded instances, including dataset, target CWE, commit hash, selected-parent hash, repository URL, normalized author timestamp, supervision role, selection status, and exclusion reason. Transition processing: `manifest_development_processing.jsonl` records original and retained lengths for each source state and diff, truncation flags, parsing diagnostics, and configuration version. Development transitions follow the budgets and retention rules in Section 1.4; obligations requiring omitted evidence remain UNRESOLVED. Membership and processing records remain fixed across suite revisions under the same validation configuration.

1.9 Evaluated Commit-Group Search Configuration

The file `config_evaluation_inference.json` records the search settings fixed before test evaluation and shared across SPI-DB* and PatchDB*. Suite acceptance follows Section 1.7, with $f_{\max} = 0.10$. Candidate groups contain at most $L_{\max} = 3$ commits, including the seed. Each member lies within history distance $H_{\max} = 5$ of the seed, measured by the shortest parent-child path in the configured history. Evidence admission follows Section 2.5: only the seed and eligible ancestors satisfying its committer-time cutoff are available. Temporal-proximity edges connect admitted comparisons whose committer timestamps differ by at most 14 days; this threshold does not restrict edges established through other relation types. The budget $B_{\max} = 10$ limits additional composite evaluations per seed, excluding singleton verification. Actual-state confirmation and seed attribution are accounted for separately. On budget exhaustion or an empty queue, search selects the evaluated non-violated record minimizing $\kappa(g)$ defined in Section 2.5, including the singleton, before evidence validation and verdict assignment.

1.10 Robustness Subset Construction and Membership

Following Section 3.6 of the main manuscript, robustness analyses reuse frozen verdicts for all 2,906 PatchDB* test seeds (892 positives and 2,014 negatives), with VERIFIED mapped to positive and other verdicts to non-positive. Assignments are independent of predictions. Weakness groups partition the test set: Memory Safety (CWE-119/125/787; 425/960 positives/negatives), Pointer Dereference (CWE-476; 135/287), Input & Lifecycle (CWE-20/190/416; 210/473), and Long-tail CWEs (122/294). Multi-group positives follow the smallest numerical CWE identifier; unmapped positives enter Long-tail. Negatives are allocated in this group order using documented quotas after sorting by author timestamp (missing or invalid values default to zero) and full commit hash. Repository domains follow primary project function: OS & Hypervisors (270/590), Multimedia (220/490), and Networking (150/330), leaving 856 seeds unassigned. Hard negatives (600) meet the documented criteria for non-security functional repairs involving bounds, pointers, memory lifecycle, or defensive control flow; remaining negatives are standard (1,414). Both tiers share all 892 positives.

The manifest `manifest_rq5_robustness_slices.jsonl` records seed identifiers, labels, memberships, assignment reasons, and policy versions. The script `reproduce_rq5_tables.py` joins memberships to frozen verdicts, computes subset metrics directly from unaggregated predictions, and checks partition coverage and domain disjointness. Slices evaluate against the overall test totals (719 true positives, 114 false positives, 1,900 true negatives, 173 false negatives); difficulty evaluations retain the shared positive pool, with 56 standard-tier and 58 hard-tier false positives.